HANDOVER · DIRECTION 3

Architecture harness: React infinite canvas with component-canvas dialogs

What to build to replace `src/mha/harnesses/arch/static/` (vanilla `app.js` + hand-rolled SVG `diagram.js`) with a React project whose canvas is editable, pans and zooms infinitely, updates live mid-turn, and opens a component's internals in its own dialog canvas rather than expanding it on the system graph.

> **THE ONE PRINCIPLE**
>
> The harness's `ArchState` stays the single source of truth. The canvas is a *projection* of it plus one client-owned overlay (node positions, annotations, sticky notes). Anything the user changes that belongs to the architecture is sent back as the tool-equivalent mutation; anything that is only about looking at the architecture never leaves the browser.

1. Scope

In scope: the whole page served by `mha arch` — one user, localhost, one session per page.

- Infinite canvas of components and connections, with pan, zoom, fit, minimap and a select/rect/arrow/text/note/freehand tool palette.
- Direct editing: move nodes (persisted), rename, edit responsibility inline; annotations and sticky notes as a canvas-only layer.
- Component internals in a **component canvas dialog** — its own pan/zoom surface, one shell per facet kind, one level deeper for a single module.
- Chat rail with streaming agent text, tool-activity lines, obligations, decisions, questions, flows; finalize gate; all connection and turn states.

Out of scope for v1: creating components or connections by drawing (read the note in §7), multiple sessions, resume UI, auth, remote access, mobile.

2. Stack

CONCERN	CHOICE	WHY
Build	Vite + React 18 + TypeScript	Static build output served by the existing harness route.
Canvas	**React Flow**	Nodes are typed React components, so a node *is* the contract sheet; gives pan/zoom/minimap/drag/handles for free and keeps positions in our store.
Not Excalidraw	—	It owns its own document model; keeping that in sync with `ArchState` costs more than it returns. Borrow the interaction feel, not the package.
Annotation layer	React Flow custom nodes (sticky, text) + one freehand SVG layer	Rides the same transform as the graph, so annotations stay pinned when you zoom.
State	Zustand: `sessionStore` (server truth) + `canvasStore` (overlay)	The split *is* the architecture: one store is replaced wholesale by every `arch_state` event, the other is never touched by the server.
Layout	ELK (layered) behind a **Tidy up** button	Never automatic: pinned nodes keep their place.
Transport	Existing SSE + POST contract, unchanged	No server work needed for v1 except the mutation endpoint in §5.

Mermaid is not used anywhere. The server already pushes structured state; rendering from it directly is what makes nodes clickable and editable.

3. Shell anatomy

Three fixed regions, no page scroll. Left **tool rail** 52px; centre **canvas**; right **rail** 380px with tabs Chat / Decisions / Questions / Flows. Top bar 44px: session goal, phase, model, repo, connection dot, **Tidy up**, and a depth switcher (L0 system · L1 internals · L2 modules) that reflects what is open rather than acting as a mode.

- Canvas overlays: a count chip and an in-progress notice top-left; minimap and a zoom pill bottom-right; a keyboard-hint strip bottom-left (⌘K commands, E open facet, ⇧E deeper, L tidy, ⎋ interrupt/close).
- Node card: kind (mono, uppercase, 9px), name 14.5px semibold, responsibility 11.5px, optional tech pills, a dot when the component owes a facet, a badge with the facet summary when it has one. `existing: true` renders dashed and dimmed.
- Connections: curved bezier, 1.4px; `async` dashed, `batch` heavier; label chips carry a canvas-

coloured backing so they stay legible over the grid.

- Chat rail scopes to the open component: while a component canvas is open, messages are prefixed with that component and the rail says so.

Tokens — palette: canvas `oklch(97.5% .003 250)`, surface white, ink `oklch(22% .02 260)`, muted `oklch(54% .015 260)`, hairline `oklch(90% .006 260)`, accent `oklch(52% .19 275)`, changed/amber `oklch(72% .15 65)`, ok `oklch(58% .12 155)`, danger `oklch(58% .17 25)`. Type: IBM Plex Sans for UI, IBM Plex Mono for ids, values and keys. One border weight (1px, 1.5px when selected), radius 8–14px, shadows only on floating surfaces.

4. The component canvas dialog

Clicking a component selects it; opening it (badge, E, or the chat rail's action) mounts a floating dialog over a dimmed canvas. **The system graph never reflows.** That is the whole point: node positions are user-owned, so growing a node in place would shove hand-placed neighbours, and internals need more room than a node's footprint anyway.

PART	CONTENTS
Header	Kind chip · name · breadcrumb once you go deeper (`order-service / pricing`) · summary ("4 modules") · **Open full canvas** · close.
Tabs	Facet-dependent: Data flow / Contract / Decisions / Failure modes for a service; Entities / Access patterns / Retention / Migration risk for a store.
Body	Its own React Flow instance — dotted field, drag, pan, zoom, fit — plus dashed **port chips** at the left and right edges naming the real inbound/outbound neighbours, so the internals stay anchored to the system graph.
Footer	Interface line (`exposes …`), invariants, and the hint strip (⎋ close, drag to rearrange).

Sizes. Docked dialog 640×470 minimum, resizable, remembers its size; **Open full canvas** promotes the same component to a full-window view with the ghost neighbours pinned at the frame edges. Same React tree, two containers.

One shell, every facet kind. The dialog is a frame; what goes inside is chosen by `facet.facet_kind`:

FACET	SUB-DIAGRAM	READS FROM
`service`	Module data-flow graph in lanes (edge → domain → persistence), typed internal edges	`modules[]`, `interface[]`
`store`	ER canvas: entity cards with fields, keys, indexes; relation edges with cardinality	`entities[]`, `access_patterns[]`
`queue`	Message flow: producers → per-message contract cards → consumers, plus DLQ	`messages[]` + connections
`api`	Endpoint table grouped by route, with auth, errors, idempotency, pagination	`endpoints[]`
`infra`	Deployment frames: a unit as a container holding the component ids it hosts	`units[]`, `state_locality`
`llm`	Task chain: prompt contract → context strategy → guardrails → fallback	`tasks[]`

A component with no facet yet opens the same dialog in an empty state: responsibility, contract, why it owes a facet, and an **Expand this component** action that posts the instruction to the agent.

> **L2 — one module deeper.** Selecting a module opens its internal chain *in the same dialog* with a breadcrumb back to L1, so the user never loses their place. This needs a schema addition — see §8. Until it lands, render the module card with its purpose and no L2 affordance; do not fake the chain.

5. Events in, mutations out

One SSE connection to `/events`; POSTs to `/input`, `/permission`, `/interrupt`. The server replays late joiners, so a refresh rebuilds everything — the canvas overlay must therefore survive reload independently (see §6).

EVENT	CANVAS + RAIL BEHAVIOUR
`ready`	Fill model, repo, run id; enable input; restore the saved overlay for this run id.
`assistant_delta`	Append to the in-flight message; render progressively (deltas are frequent — buffer to one paint per frame).
`harness_event` tool calls	One compact activity line per call, keeping today's glyph vocabulary (+ `component`, → `connect`, ◆ `decide`, ▸ `expand`, ⌕ `kg_query`).
`arch_state`	Replace server truth wholesale. Diff by id against the previous snapshot: new nodes fade+slide in at an auto-placed spot, changed nodes and edges get the amber ring for ~2s, removed nodes fade out. Never re-fit and never move a node the user has placed. Refresh the drawer lists, obligations and any open dialog in place.
`permission_request`	`toplevel_approval` → Approve banner; `finalize` → Finalize / Request changes. Non-blocking: replying in the composer routes as feedback.
`turn_end`	Close the message, show status (completed / interrupted / error), re-enable input.
`error` · bye / drop	Non-fatal notice with Retry; disconnected veil over the canvas with input disabled (canvas stays pannable so the user can still read it).

Edits go out as mutations. Apply optimistically, then POST. On failure, roll the node back and drop an inline notice in the rail — never leave the canvas showing something the harness does not believe.

USER ACTION	SENT AS	NOTES
Move a node	nothing	Client overlay only; marks the node pinned so Tidy up leaves it alone.
Rename · edit responsibility	`component` mutation (post-approval: `amend_toplevel`)	Ids are immutable — a rename changes `name` only. Show the pending mutation in the inspector footer with an Apply affordance.
Sticky note · annotation · freehand	nothing	Canvas-only, never in the handoff bundle. Offer “send as a question” to promote one into `ask`.
Expand a component	`/input` instruction	Same phrasing today's page uses; the agent runs `expand` and the dialog fills in live.
Interrupt · approve · finalize	`/interrupt`, `/permission`	Unchanged from today.

Server work needed: one endpoint that accepts a structured mutation and applies it through the same validation path the model's tools use (so `trace`, kind-conditional fields and the amendment audit trail all still hold), returning the new state or a validation error. Reject silently-invalid edits rather than accepting them into state.

6. The canvas overlay

Persist per run id (`localStorage` key `mha_arch_canvas:<run_id>`): node positions and pinned flags, viewport transform, sticky notes and annotations, dialog size, rail tab, chat height. Restore on `ready`.

- Placement for an unpinned new node: run ELK on the unpinned subset only, with pinned nodes as fixed obstacles — so the graph stays readable without ever moving hand-placed cards.
- Ids are stable and immutable, so overlay entries key cleanly off `component.id`; drop entries whose id disappears.
- Never write to storage keys the page didn't create; keep today's `mha_arch_chat_h` behaviour working.

7. States to build

Each of these is a real state with its own screen, not an afterthought: connecting / first load with the initial turn already streaming · agent turn in progress (text streaming while nodes land) · idle · empty architecture · user editing with a pending mutation · component dialog open (filled, empty-facet, and filling live mid-turn) · L2 open · finalize pending · interrupted turn · turn error · disconnected · finalized read-only.

Read-only after finalize means: canvas still pans, zooms and opens dialogs; every edit affordance and the composer are gone; the rail shows the bundle paths and `mha code` as the next step.

8. Schema delta for L2 (decision needed)

L1 needs no schema change — every facet kind already carries the fields its sub-diagram draws. L2 does: `Module` is currently name + purpose, which is not enough to draw a chain. Two options, pick one before building L2:

- **Author it.** Add optional `steps: list[ModuleStep]` and `reads: list[str]` to `Module`, where a step is name + purpose + optional note, filled by a scoped `expand_module` tool. Deterministic and immediate; costs the model a few more tokens per risky module.
- **Derive it.** Leave the schema alone and build L2 from the knowledge graph once code exists. Free at design time, but empty on greenfield — which is exactly when the user wants it.

Recommendation: author it, gated on risk the same way facet obligations are — only modules on a critical flow, or holding rules that are expensive to get wrong, owe an L2. Otherwise the model writes detail nobody reads.

9. Build order

PHASE	SHIPS	DONE WHEN
1 · Parity	Vite shell, SSE client, both stores, React Flow canvas, rail, composer, all connection/turn states	A full session runs end to end with no regression against today's page.
2 · Canvas	Overlay persistence, drag + pin, Tidy up, minimap, tool palette, sticky notes, annotations, ⌘K	A mid-turn state push never moves a node the user placed.
3 · Editing	Inline rename + responsibility, pending-mutation UI, mutation endpoint, rollback path	A rejected edit rolls back visibly and the state stays consistent.
4 · Dialog L1	Dialog shell, port chips, service + store + queue sub-diagrams, empty-facet state, full-canvas promotion	Opening a component during a live expand fills in without closing.
5 · Dialog L2	Schema delta, `expand_module`, module chain view, breadcrumb	A builder can read one module's rules without opening the repo.
6 · Polish	api + infra + llm sub-diagrams, flow playback, canvas PNG export	Every facet kind has a sub-diagram; nothing falls back to a bare list.

10. Acceptance checks

- Move three nodes, then let the agent add two more: the three stay exactly where they were put.

- Open a component dialog mid-turn: it fills as tools land, and the system graph behind it does not shift by a pixel.
- Rename a component: the id in every connection, flow and the bundle is unchanged.
- Refresh the page mid-session: transcript, state, positions, notes and viewport all come back.
- Kill the server: disconnected veil, input disabled, canvas still readable and pannable.
- Finalize: canvas read-only, bundle paths shown, no edit affordance anywhere.
- Zoom to 30% and to 300%: labels stay legible or hide cleanly; no label ever sits under a card.

11. Open questions

- Should drawing a new component or connection on the canvas be allowed in v1, or stay agent-only? (Today's answer: agent-only; the user edits what exists.)
- Should the dialog be dockable into the right rail for side-by-side reading, or stay floating?
- Do annotations belong in the session log at all — useful history, or noise the harness shouldn't carry?
- L2: author or derive (§8) — needs a decision before phase 5.

Reference mocks: `Arch Canvas Redesign.dc.html` — **3a** system canvas with the dialog open, **3b** dialog at L2, **3c** the store variant of the same shell; **2b** shows the full-canvas promotion. Baseline being replaced: **0a**.